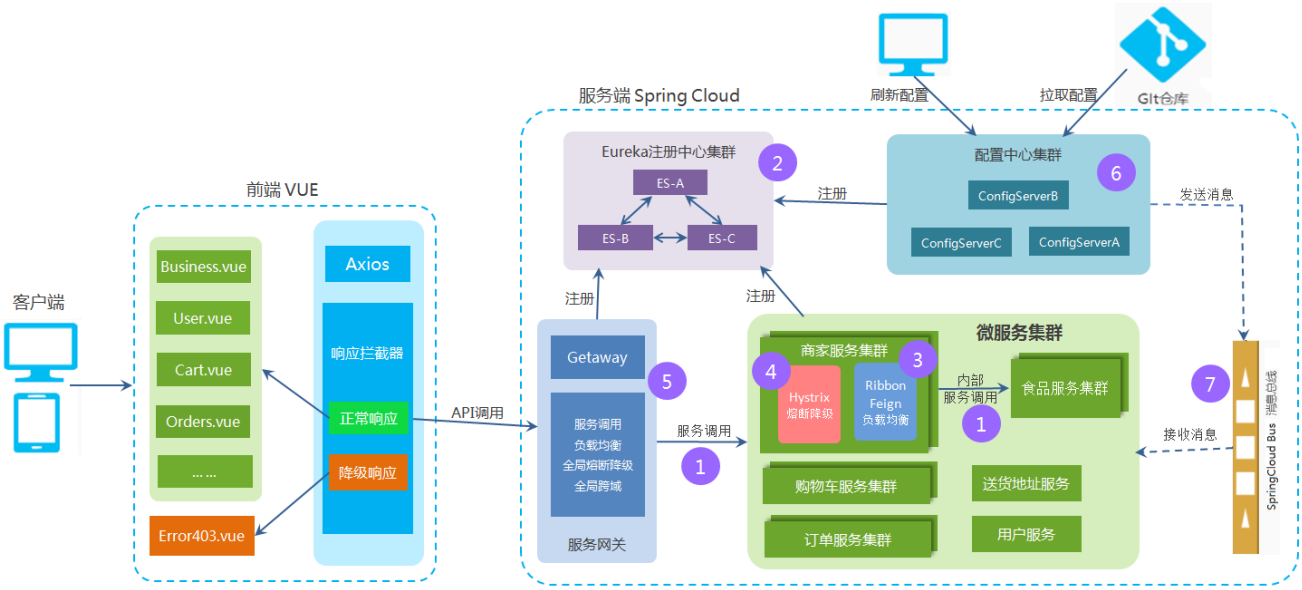
1.添加Feign服务调用及负载均衡

本阶段项目完成总架构图中的第三阶段，添加Feign的内部服务调用及负载均衡。



1.1.创建食品服务集群

在父工程下，创建 Maven Module 子工程（工程名：food_server_10201；Packaging：jar）。此工程的内容与food_server_10200工程一致，除了端口号。所以修改food_server_10201工程的application.yml文件：

```yaml
server:
    port: 10201

spring:
    application:
        name: food-server
    #业务配置
    datasource:
        username: root
        password: 123
        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
        driver-class-name: com.mysql.jdbc.Driver

#业务配置
logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug
#业务配置
mybatis:

    mapper-locations: classpath:mapper/*.xml
```

```
    type-aliases-package: com.neusoft.po

#eureka配置
eureka:
    client:
        service-url:
            #将自己注册给Eureka Server集群
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
    instance:
        prefer-ip-address: true        #使用ip地址向Eureka注册
        instance-id: ${spring.cloud.client.ip-address}:${server.port} #自定义微服务实例ID
```

> 工程启动后，在Eureka工作台中检查食品服务集群是否注册成功。

1.2.添加基于Feign的服务调用

在商家微服务（business_server_10300）工程中使用Feign进行服务调用及负载均衡。

1.2.1.导入Feign依赖

在business_server_10300工程中添加Feign的依赖

```
<!--SpringCloud整合的openFeign -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

1.2.2.主启动类添加Feign注解

在主启动类上添加开启Feign支持的注解

```
@SpringBootApplication
//开启Spring Cloud Feign的支持功能
@EnableFeignClients
public class MyApplication {

    //RestTemplate不需要了
    /*
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }
    */
```

```
    //... ...
}
```

1.2.3.配置调用接口

创建在Feign中调用微服务的接口

```
package com.neusoft.feign;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.neusoft.po.CommonResult;

//指定调用的食品微服务名
@FeignClient(name="food-server")
public interface FoodFeignClient {

    @GetMapping("/FoodController/listFoodByBusinessId/{businessId}")
    public CommonResult listFoodByBusinessId(@PathVariable("businessId") Integer businessId);
}
```

2.2.4.使用Feign接口进行调用

在控制器BusinessController中注入Feign接口，并使用它进行服务调用

```
@CrossOrigin("*")
@RestController
@RequestMapping("/BusinessController")
public class BusinessController {

    @Autowired
    private BusinessService businessService;
    @Autowired
    private FoodFeignClient foodFeignClient;

    @GetMapping("/listBusinessByOrderTypeId/{orderTypeId}")
    public CommonResult<List> listBusinessByOrderTypeId(@PathVariable("orderTypeId") Integer
orderTypeId)
            throws Exception {
        List<Business> list = businessService.listBusinessByOrderTypeId(orderTypeId);
        return new CommonResult(200, "success", list);
    }

    @GetMapping("/getBusinessById/{businessId}")

    public CommonResult<Business> getBusinessById(@PathVariable("businessId") Integer
```

```
businessId) throws Exception {
        Business business = businessService.getBusinessById(businessId);
        //在商家微服务中调用食品微服务
        CommonResult<List> result = foodFeignClient.listFoodByBusinessId(businessId);
        System.out.println(result.getMessage());
        if (result.getCode() == 200) {
            business.setFoodList(result.getResult());
        }
        return new CommonResult(200, "success", business);
    }
}
```

1.2.5.测试

启动服务，测试服务调用，同时也可以测试负载均衡（Feign也封装了Ribbon的负载均衡功能）。